

HookSuite — Bienvenida al proyecto

Léelo antes de abrir el manual técnico. Te dará el contexto que necesitas para entender por qué haces lo que haces.

Qué es HookSuite

HookSuite es una herramienta de auditoría de seguridad web que estamos construyendo desde cero. Su función es analizar aplicaciones web en busca de vulnerabilidades de forma automatizada, combinando tres cosas:

- **Interceptación de tráfico HTTP** — capturar todo lo que el navegador envía y recibe
- **Automatización con IA** — un navegador que navega, ataca y detecta vulnerabilidades solo
- **Dashboard web** — una interfaz desde la que controlar todo y ver los resultados en tiempo real

El resultado final tiene que parecerse funcionalmente a Burp Suite, una herramienta profesional de pentesting que usan los analistas de seguridad en el mundo real. Nosotros la construimos desde cero, la desplegamos en un servidor real y la presentamos funcionando el día de la entrega.

Por qué lo estamos construyendo

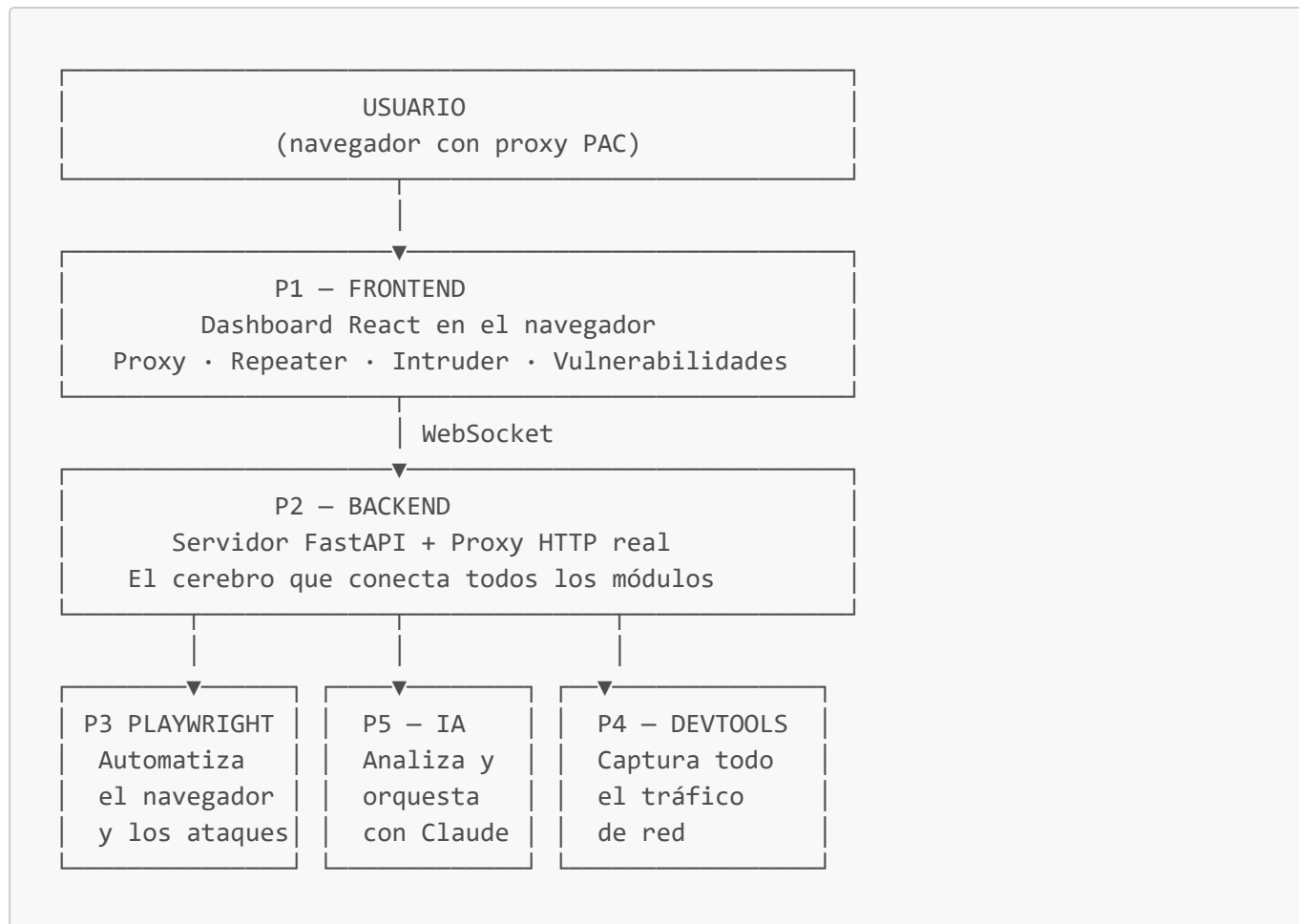
Es la Práctica 1 del Módulo de Ciberseguridad Avanzada. La entrega es el **25 de Mayo de 2026** y hay cuatro cosas que se evalúan:

Qué evalúan	Peso
Que la herramienta funciona de verdad	30%
Que el dashboard se ve y se usa bien	15%
Que está desplegada en Hetzner y accesible desde internet	15%
Que el informe PDF está bien hecho	20%
Que el repositorio GitHub está limpio y organizado	10%
Que el road map de mejoras está bien elaborado	10%

La presentación es en vivo, 10 minutos por grupo, demostrando la herramienta funcionando en tiempo real ante el profesor. No hay margen para que algo falle el día de la demo.

Cómo está organizado el equipo

Somos 5 personas y cada una construye un módulo independiente. Los módulos están diseñados para poder desarrollarse en paralelo y conectarse entre sí en la semana 3.



P1 Frontend construye todo lo que el usuario ve: el dashboard, los paneles, los formularios, las alertas. Es la cara visible del proyecto.

Bloque 01 — Estructura base del proyecto React

- Paso 1 — Clonar el repositorio y crear tu rama de trabajo
- Paso 2 — Inicializar el proyecto React con Vite
- Paso 3 — Crear la estructura de carpetas
- Paso 4 — Crear los componentes de sistema de diseño base
- Paso 5 — Crear el layout principal con React Router
- Paso 6 — Crear los mocks de datos
- Paso 7 — Primer commit

Bloque 02 — Módulo Proxy / Interceptor

- Paso 8 — Crear el hook de WebSocket
- Paso 9 — Crear el panel de peticiones interceptadas
- Paso 10 — Crear la vista detalle de petición
- Paso 11 — Crear el onboarding del PAC
- Paso 12 — Ensamblar la página Proxy completa

Bloque 03 — Módulo Repeater

- Paso 13 — Crear el editor de petición HTTP
- Paso 14 — Crear el panel de respuesta con diff
- Paso 15 — Ensamblar la página Repeater

Bloque 04 — Módulo Intruder

- Paso 16 — Crear la configuración de ataque, panel de progreso y tabla de resultados

Bloque 05 — Panel de vulnerabilidades y red en tiempo real

- Paso 17 — Crear el dashboard de vulnerabilidades
- Paso 18 — Crear el panel de red en tiempo real

Bloque 06 — Módulo de Utilidades

- Paso 19 — Crear Encoder/Decoder, Hash Generator, Regex Tester y Payload Generator

Bloque 07 — Integración con datos reales y UX final

- Paso 20 — Conectar con el backend de P2
- Paso 21 — Pulir la UX: estados de carga y errores
- Paso 22 — Preparar el build de producción
- Paso 23 — Abrir el Pull Request a main

Bloque 08 — Documentación del módulo

- Paso 24 — Capturas para el informe PDF
- Paso 25 — Manual de uso del frontend
- Paso 26 — Script de demo para la presentación

P2 Backend construye el servidor que conecta todo. Sin él, ningún otro módulo puede comunicarse con los demás. Es el rol más crítico.

Bloque 01 — Servidor base y WebSockets

- Paso 1 — Clonar el repositorio y crear tu rama de trabajo
- Paso 2 — Configurar el entorno Python
- Paso 3 — Crear la estructura de carpetas del backend
- Paso 4 — Crear el servidor FastAPI base
- Paso 5 — Crear la gestión de sesiones

Bloque 02 — Proxy HTTP interceptor

- Paso 6 — Crear los modelos Pydantic
- Paso 7 — Crear el servicio del proxy HTTP
- Paso 8 — Crear el archivo PAC y el endpoint de verificación
- Paso 9 — Implementar el proxy TCP real con mitmproxy
- Paso 10 — Conectar el proxy con el WebSocket

Bloque 03 — Repeater

- Paso 11 — Crear el endpoint de reenvío manual
- Paso 12 — Commit del bloque Repeater

Bloque 04 — Intruder / Motor de fuzzing

- Paso 13 — Crear la biblioteca de payloads
- Paso 14 — Crear el motor de fuzzing
- Paso 15 — Crear los endpoints del Intruder

Bloque 05 — Utilidades del backend

- Paso 16 — Crear los endpoints de utilidades (hash, encode, regex)

Bloque 06 — Receptor de paquetes de red

- Paso 17 — Crear el endpoint receptor de paquetes de DevTools

Bloque 07 — Despliegue en Hetzner

- Paso 18 — Crear el Dockerfile del backend
- Paso 19 — Aprovisionar el servidor Hetzner
- Paso 20 — Desplegar HookSuite en Hetzner
- Paso 21 — Configurar el subdominio

Bloque 08 — Documentación y entrega

- Paso 22 — Documentar la API con FastAPI
 - Paso 23 — Escribir el diagrama de arquitectura técnica
 - Paso 24 — Escribir la guía de despliegue paso a paso
 - Paso 25 — Capturas para el informe PDF
 - Paso 26 — Documentar el módulo backend
-

P3 Playwright construye el navegador automatizado que navega webs, descubre formularios y lanza ataques sin que nadie tenga que hacer clic.

Bloque 01 — Setup y configuración de Playwright

- Paso 1 — Clonar el repositorio y crear tu rama de trabajo
- Paso 2 — Configurar el entorno Python
- Paso 3 — Levantar el entorno de pruebas DVWA
- Paso 4 — Crear la estructura de carpetas
- Paso 5 — Crear el gestor del navegador con optimización de velocidad

Bloque 02 — Navegación y reconocimiento automático

- Paso 6 — Crear el sistema de autenticación
- Paso 7 — Crear el spider / crawler
- Paso 8 — Crear el descubridor de formularios
- Paso 9 — Crear el fingerprinting de tecnologías
- Paso 10 — Commit del bloque de reconocimiento

Bloque 03 — Automatización de interacciones y ataques

- Paso 11 — Crear el módulo de automatización de ataques
- Paso 12 — Crear el sistema de reporte al backend
- Paso 13 — Crear el punto de entrada principal

Bloque 04 — Orquestación con el módulo de IA

- Paso 14 — Crear el receptor de instrucciones de la IA
- Paso 15 — Prueba de flujo completo de ataque

Bloque 05 — Demo y documentación

- Paso 16 — Optimización de velocidad para la demo
 - Paso 17 — Preparar el script de demo
 - Paso 18 — Abrir el Pull Request
 - Paso 19 — Capturas para el informe PDF
 - Paso 20 — Documentación del módulo
-

P4 Chrome DevTools construye el módulo que captura todo el tráfico de red del navegador en tiempo real, lo analiza y lo pasa al módulo de IA.

Bloque 01 — Setup de Chrome DevTools Protocol

- Paso 1 — Clonar el repositorio y crear tu rama de trabajo
- Paso 2 — Configurar el entorno Python
- Paso 3 — Levantar DVWA
- Paso 4 — Crear la estructura de carpetas
- Paso 5 — Crear el lanzador de Chrome
- Paso 6 — Crear el cliente CDP
- Paso 7 — Crear el constructor de paquetes de red
- Paso 8 — Commit del bloque 01

Bloque 02 — Captura y análisis de tráfico

- Paso 9 — Crear el analizador de red
- Paso 10 — Crear el analizador de consola
- Paso 11 — Crear el reporter para enviar paquetes a P2
- Paso 12 — Commit del bloque 02

Bloque 03 — Punto de entrada principal

- Paso 13 — Crear el main.py

Bloque 04 — Integración con P2 y P5

- Paso 14 — Verificar integración con P2
- Paso 15 — Preparar datos para P5
- Paso 16 — Commit de integración

Bloque 05 — Road Map Práctica 2

- Paso 17 — Elaborar el road map con 10 mejoras y estimaciones

Bloque 06 — Documentación y entrega

- Paso 18 — Abrir el Pull Request

- Paso 19 — Capturas para el informe PDF
 - Paso 20 — Documentación técnica del módulo
-

P5 IA + GitHub construye el cerebro del sistema: el módulo que analiza los datos capturados, detecta vulnerabilidades con Claude y orquesta los ataques. Además es el responsable de que el repositorio esté limpio y de que el informe PDF final quede profesional.

Bloque 01 — GitHub: estructura y gobierno del repositorio

- Paso 1 — Crear y estructurar el repositorio
- Paso 2 — Crear las ramas de trabajo para cada rol
- Paso 3 — Configurar la protección de la rama main
- Paso 4 — Crear el pipeline de GitHub Actions
- Paso 5 — Establecer la rutina de revisión semanal

Bloque 02 — Módulo de IA: análisis de vulnerabilidades

- Paso 6 — Configurar el entorno Python del módulo de IA
- Paso 7 — Crear el cliente de la API de Anthropic
- Paso 8 — Crear el prompt de análisis de paquetes de red
- Paso 9 — Crear el clasificador de vulnerabilidades
- Paso 10 — Commit del bloque 02

Bloque 03 — Módulo de IA: orquestación de ataques

- Paso 11 — Crear el orquestador del ciclo de ataque
- Paso 12 — Crear el punto de entrada del módulo IA

Bloque 04 — Informe PDF: cohesión y maquetación

- Paso 13 — Establecer la fecha límite interna de documentación
- Paso 14 — Instalar Typst y preparar la plantilla del informe
- Paso 15 — Redactar las secciones transversales del informe
- Paso 16 — Integrar las secciones de cada rol
- Paso 17 — Generar el PDF final

Bloque 05 — Gestión de Pull Requests y entrega final

- Paso 18 — Revisar y mergear los Pull Requests
 - Paso 19 — Verificar el estado final del repositorio
 - Paso 20 — Abrir tu propio Pull Request
 - Paso 21 — Documentación del módulo de IA
-

Las 4 semanas del proyecto

Semana 1 — Cimientos Cada módulo arranca de forma independiente. Al final de la semana tienes tu entorno funcionando, tu primera funcionalidad básica operativa y tu primer commit en el repositorio.

Semana 2 — Primera conexión Los módulos empiezan a hablar entre sí. El frontend conecta con el backend por WebSocket. DevTools empieza a enviar paquetes al backend. La IA recibe sus primeros datos reales.

Semana 3 — Integración El sistema funciona de punta a punta. Playwright recibe instrucciones de la IA, ejecuta ataques y devuelve resultados. El dashboard muestra vulnerabilidades detectadas en tiempo real. Es la semana más intensa y la más crítica.

Semana 4 — Presentación El servidor Hetzner está corriendo, el informe PDF está terminado, el repositorio GitHub está limpio y la demo está ensayada. Los días 23, 24 y 25 son solo para pulir y ensayar, no para desarrollar.

Cómo vamos a trabajar

Cada uno en su rama. Nunca trabajas directamente en `main`. Tu rama se llama `feature/[tu-modulo]`. Cuando tienes algo estable, abres un Pull Request y P5 lo revisa y mergea.

Los commits tienen formato fijo. Esto es importante para que el historial del repositorio sea legible y el profesor lo valore. El formato es:

```
tipo(modulo): descripción en imperativo presente
```

Ejemplos:

```
feat(frontend): panel de peticiones interceptadas con filtros
fix(backend): corregir timeout en proxy HTTP
docs(ia): documentar prompts de análisis de SQLi
```

Cuando llegues a un punto de integración con otro módulo, el manual te lo indica. Hay bloques marcados con 🍷 que te dicen exactamente a quién contactar, qué pedirle y qué necesitas de él para continuar. No tienes que bloquearte esperando, puedes avanzar con datos simulados (mocks) hasta que el compañero tenga su parte lista.

La IA es tu copiloto. El manual incluye prompts listos para copiar en Claude Code o Claude.ai en los puntos técnicos más complejos. Úsalos. No pierdas tiempo investigando algo que la IA puede resolver en 30 segundos.

Si algo no funciona y llevas más de 30 minutos bloqueado, pide ayuda. No es rendirse, es eficiencia. Este proyecto tiene fecha límite real.

El día de la demo

La presentación dura 10 minutos. El flujo que demostraremos es:

1. Abrir HookSuite desde el navegador con el proxy configurado
2. Navegar por la web objetivo y ver las peticiones interceptándose en tiempo real

3. Seleccionar una petición interesante y modificarla en el Repeater
4. Lanzar un ataque automático desde el Intruder
5. Mostrar la vulnerabilidad detectada por la IA en el panel de alertas

Todo esto tiene que funcionar en vivo, sin fallos, en 10 minutos. Eso significa que el flujo completo tiene que estar ensayado al menos dos veces antes del día 25 y tiene que haber una grabación de backup por si algo falla.

Una última cosa

Este proyecto es técnicamente ambicioso. Cinco módulos, cuatro semanas, una demo en vivo. Va a haber momentos en los que algo no funcione como esperabas. Eso es normal y forma parte del proceso.

Lo que hace que proyectos así salgan bien no es que todo funcione a la primera, sino que el equipo reacciona rápido cuando algo falla y tiene suficiente margen de tiempo para hacerlo. Ese margen lo crea cada uno siendo disciplinado con su trabajo durante las primeras semanas.

Tu manual técnico tiene todo lo que necesitas. Empieza por el Paso 1 y sigue en orden.

Proyecto HookSuite — Módulo de Ciberseguridad Avanzada 2026